

Attention Mechanism for Human Motion Prediction

Amal Fahad Al-aqel

Department of Computer Science
Umm Al-Qura University
Makkah, Saudi Arabia
Amal.alagel@gmail.com

Murtaza Ali Khan

Department of Computer Science
Umm Al-Qura University
Makkah, Saudi Arabia
makkhan@uqu.edu.sa

Abstract— Human motion prediction aims to forecast the most likely future frames of motion conditioned on a given sequence of frames. Because of its importance to many applications especially robotics, it has received a lot of interest and has become an active area of research. Since humans are very flexible in nature, human motion prediction is very challenging. Recently, deep learning methods have been dominant in many tasks due to their successful results. Particularly, Recurrent Neural Networks (RNNs) have shown excellent performance on human motion prediction task and other tasks that depend on sequential data, where preserving the order of the sequence items is crucial. The well-known Sequence-to-Sequence (Seq2Seq) architectures have been used for sequence learning where two RNNs namely the encoder and the decoder work cooperatively to transform one sequence to another. In the context of neural machine translation, the use of attention decoders yields state-of-the-art results. This work employs a simple but efficient Seq2Seq model with attention decoder. Both encoder and decoder are trained jointly to predict 15 different categories of human motion. Our experiments have shown that the attention decoder clearly outperforms earlier methods and achieves state-of-the-art results in the short-term (< 500ms) motion prediction task. Contrary to earlier methods that show progressive deterioration as the time of prediction increases, our model shows high quality long-term (> 500ms) motion prediction which stays as high even after 1000ms of prediction.

Keywords—component; Attention; decoder; encoder; RNN; human motion prediction

I. INTRODUCTION

Humans are blessed with a remarkable ability to make accurate short-term predictions about their surroundings based on past observations [1]. Crossing a crowded street would be very challenging without our capacity of anticipating pedestrian's most likely actions in the near future [2]. Given a set of 3D poses or skeletons, the goal of human motion prediction is to forecast the most likely future frames of motion based on the given sequence [3]. Human motion prediction is essential for achieving the goal of robotic intelligence. Many situations involve human-robot interaction such as handshaking during socialization or handing tools to a surgeon during an operation. For the interaction to be successful, the robot is supposed to recognize and forecast limbs' pose and position precisely so that it can provide a rapid and proper response [3]. Human motion forecasting proved to be important for plenty of tasks e.g. action detection, action recognition, action analysis, etc. [4]. Because of humans' flexibility, they can perform complex movements which are governed by the physical laws and the intentions of the moving person. Thus, human motion is

inherently stochastic and non-deterministic which makes the task of modeling human motion very challenging. [2]. Many future poses are of high probability for the same set of observed sequence poses [4]. Generally, the literature refers to the task of long-term prediction as motion generation which is of special interest to the computer graphics community. On the other hand, the task of short-term prediction is commonly referred to as motion prediction which mostly concerns computer vision's community [4]. The former is harder to validate quantitatively, therefore a qualitative metric, specifically human judgment is crucial while the latter can be validated quantitatively typically by measuring the mean-squared error in the angle space [2].

This work uses a Seq2Seq model with attention decoder in the task of human motion prediction. Our model surpasses earlier models in both long-term and short-term motion prediction tasks.

A. Recurrent Neural Networks and Human Motion Prediction

Deep learning methods have proved to be successful for many learning tasks [5]. Particularly, Recurrent neural networks (RNNs) have shown good performance in motion prediction and generation [6] [2]. Unlike traditional methods which require expert knowledge about human motion upfront, RNNs like other deep learning methods can be trained to automatically generalize to novel data [7]. RNNs excel at processing sequential data because of their ability to capture temporal dependencies between elements of a sequence. Unlike traditional feedforward neural networks (FNNs), RNNs have internal loops to persist information allowing them to remember the context of previously seen inputs. FNNs process inputs at each iteration independently i.e. they lack any form of memory. However, RNNs are capable of handling variable-length sequences by using a recurrent unit such that the hidden state $h^{(t)}$ at timestep t depends on the previous one i.e. $h^{(t-1)}$ as shown in (1) [14]:

$$h^{(t)} = g_1(W_h h^{(t-1)} + W_x x^{(t)}) \quad (1)$$

Where $x^{(t)}$ is the current input, W_h and W_x are weight matrices, and g_1 is a non-linear activation e.g. Tanh, Sigmoid or ReLu. Optionally, RNN may produce an output $y^{(t)}$ at each timestep or only at the last timestep. $y^{(t)}$ is given by (2) [14]:

$$y^{(t)} = g_2(W_y a^{(t)}) \quad (2)$$

Where W_y is the weight matrix and g_2 is a non-linear activation. Equations (1) and (2) describe a traditional vanilla RNN.

B. Vanishing/Exploding Gradients

Almost none of the RNNs successes were achieved by vanilla RNNs. Rather they were achieved by RNNs with sophisticated recurrent units. Training vanilla RNNs is hard due to the vanishing/exploding gradients problem [15]. During backpropagation, gradients either become extremely small (vanish) very often or become extremely large (explode) rarely but with much worse impact. This problem was tackled by researchers either by improving the learning algorithm itself e.g. using gradient clipping technique or by modifying the architecture of the network by designing more sophisticated recurrent units e.g. long short-term memory (LSTM) and gated recurrent unit (GRU). LSTM and GRU are the most effective sequence models in use today [8]. They are designed to capture long-term dependencies by a using gating mechanism to regulate the flow of information. LSTM was proposed by [16] and has been improved by many others ever since. More recently, a simpler version of LSTM namely GRU was proposed by [17]. LSTM and GRU are well documented in the literature. Gradient clipping is a simple technique where the gradient is clipped when it reaches some threshold.

C. Bidirectional Recurrent Neural Networks

Some applications may require information not only from the past of the input sequence but also from the future e.g. speech recognition. Bidirectional RNN was invented to fulfill that need [9]. It uses a forward and a backward RNNs. The former takes the sequence in the original order and the latter takes the sequence in the reversed order [8]. Both produce hidden states as shown in (3) and (4) [14]:

$$h_f^{(t)} = g_1 (W_h^f h^{(t-1)} + W_x^f x^{(t)}) \quad (3)$$

$$h_b^{(t)} = g_1 (W_h^b h^{(t-1)} + W_x^b x^{(t)}) \quad (4)$$

Where $h_f^{(t)}$ and $h_b^{(t)}$ are the forward and backward hidden states respectively.

D. Sequence to sequence architectures

Sequence to sequence (Seq2Seq) or Encoder-Decoder architecture was proposed first by [19] and [20] to map an input sequence to an output sequence where both can be of variable-lengths. The basic Seq2Seq architecture consists of two RNNs namely the encoder and the decoder. The encoder is trained to process the input sequence step by step to produce a fixed-length vector (context vector) and the decoder is trained to recover the output sequence from the context vector. Ideally, the context vector summarizes the whole input sequence. At every encoding step, the encoder takes the current element $x^{(t)}$ of the input sequence x , and the previous encoder's hidden state $h_e^{(t-1)}$ to produce the current hidden state $h_e^{(t)}$ as shown in (5) where $h_e^{(0)}$ is initialized to 0. Upon finishing, the last encoder's hidden state represents the context vector that is given as the initial hidden state to the decoder. At every step of decoding, the decoder consumes its previous hidden state $h_d^{(t-1)}$ which is updated at every step as shown in (6). In the context of machine translation, $y^{(0)}$ is start-of-sentence token <SOS>, whereas in motion prediction $y^{(0)}$ is the last frame of the input sequence. In the subsequent steps of decoding, $y^{(t-1)}$ could be the decoder's own

prediction of the previous step or the ground-truth of the previous step. Feeding the decoder with its predictions at every training step is known as sampling while feeding it with the ground-truth is known as teacher forcing. A combination of both is known as scheduled sampling. During testing, the decoder always sees its own predictions. Authors of [2] favored sampling over teacher forcing as they found that the former achieves good results while the latter results in a network that is unable to recover from its own mistakes.

$$h_e^{(t)} = f(h_e^{(t-1)}, x^{(t)}) \quad (5)$$

$$h_d^{(t)} = f(h_d^{(t-1)}, y^{(t-1)}) \quad (6)$$

E. Attention Mechanism

The notion of attention has gained popularity in diverse learning applications [22]. In the context of machine translation, the work of [21] was the first to exploit the power of attention [22]. Authors of [21] argue that the use of a fixed-length context vector in Seq2Seq models is a bottleneck, especially for long sequences. Instead of putting the burden on the context vector alone to encode the entire input sequence, the attention allows the decoder at every step to selectively focus on (attend to) different parts of the input sequence that are more relevant to the current part of the output sequence. Unlike attention decoder, classic decoder ignores all hidden states of the encoder and uses the last hidden state (context vector) alone during decoding thus not making use of precious information about the input sequence encoded by the overlooked hidden states. On the other hand, attention decoder takes all hidden states into consideration. Based on the architecture used by [21], the work of [22] suggested a simpler version of attention which has achieved the state-of-the-art results in the neural machine translation task. The attention decoder takes its current hidden state h_t (target state) concatenated with the context vector c_t to get new state $\tilde{h}_t$. c_t is obtained by multiplying attention weights with all encoders' hidden states h_s (source). There are several ways to obtain attention weights as described in [22].

II. LITERATURE REVIEW

This section briefly reviews the recent works that employ deep learning to predict human motion with a special focus on RNNs. The reviewed works mainly rely on motion data acquired from motion capture (MOCAP). The literature refers to motion prediction as a short-term prediction when it is less than 500ms, which is the main concern of this research. Otherwise, motion prediction that exceeds 500ms is referred to as a long-term generation or synthesis. For skeleton-based tasks, deep learning has outperformed traditional methods [3]. An early work of [13] suggested a conditional restricted Boltzmann machine (CRBM) to model motion where sampling is needed for inference [2]. The model was experimented with few motions including walking, jogging and running [7]. Encoder-Recurrent-Decoder (ERD) architecture was proposed by [6] which is an RNN that integrates a nonlinear encoder and a decoder before and after the recurrent layers. To ease accumulated errors during training, authors of [6] used noise scheduling by adding random noise gradually to the input data during training. Although noise scheduling has alleviated the problem, noise scheduling is hard to tune in practice [2]. Another architecture was proposed in the

same study, namely 3-layer long short-term memory network (LSTM-3LR) [6]. Both architectures consist of concatenated Long Short-Term Memory (LSTM) units, but the former has a non-linear space encoder for data preprocessing. Recently, authors of [2] have further extended the work of [6] by proposing a simpler architecture with one Gated Recurrent Unit (GRU) instead of using the more complicated LSTM unit which was used in previous works [6, 13]. To address the problem of the discontinuity between the last frame of the input sequence and the first frame of the predicted sequence, authors of [2] have modeled velocities instead of absolute joint angles by deploying residual connections which leads to smoother and more accurate predictions. In another study [13], authors have combined high-level spatio-temporal graphs (st-graphs) with RNNs by developing a structural RNN (SRNN) which transforms an st-graph into a trainable and scalable RNNs mixture. SRNN aims to exploit the power of RNNs at modeling sequential data and to capture spatio-temporal relations between joints. As in [6], noise scheduling was employed to reduce the effect of accumulated errors. As a result, SRNN generates plausible motions for actions like eating. For more challenging actions like discussion, the generated motion is still realistic [2]. In the previously mentioned works, the traditional Euclidean loss was used to measure the distance between the prediction and the ground truth during training. However, a novel loss function, i.e. the geodesic loss, was proposed in [1]. Unlike Euclidean distance, the geodesic distance exploits the geometric structure of 3D rotations and thus, avoids inaccuracies in predictions. Unlike Euclidean loss, geodesic loss doesn't converge to mean pose after longer time periods. Inspired by generative adversarial networks (GANs), the authors introduced two global discriminators to validate the predicted motion while casting the predictor as a generator. The generator is a decoder-encoder network and both discriminators are RNNs. The first global discriminator is a fidelity discriminator that validates the overall plausibility of the predicted motion and the second is a continuity discriminator that validates the coherence of the predicted sequence with the input sequence. The work of [1] has achieved state-of-the-art results in human motion prediction. Recently, the work of [5] uses Quaternions to represent joint angles, which was overlooked by previous works. As Euler angles suffer from singularities and discontinuities, using them usually leads to the notorious problem of exploding gradients which makes the training very difficult or even impossible. Alternatively, Exponential maps were used in previous works to alleviate these issues to some degree. In addition to using Quaternions, the authors suggested a loss function that penalizes absolute joint position errors instead of joint angle errors. Moreover, both convolutional and recurrent architectures were investigated and evaluated on short-term prediction and long-term generation.

III. DATASET

Human3.6 (H3.6) dataset [10, 11] is used in the experimentation of this work as in previous works. Currently, it is the largest publicly available motion dataset with 3.6 million 3D poses. It has 50Hz framerate (50 frames/sec) and it provides motion data for 7 professional actors performing 15 different activities. It uses a skeleton with 32 joints, and it provides motion data in several formats e.g. image data, scanner data,

pose data, etc. Pose data includes two parametrizations namely joint positions and joint angles. Joint angles of pose data of the dataset used by this work are given in exponential maps representation. Following the works of [2, 5, 6, 13], this work uses the same preprocessed version of H3.6 dataset provided by [13]. The preprocessing resembles that of [12]. As in earlier works, subject 5 is used for testing and the remaining subjects are used for training.

IV. METHODOLOGY

This work uses a Seq2Seq model consisting of an encoder and an attention decoder (At-Dec), each of which has a single GRU layer. Upon finishing encoding the source sequence, the encoder produces two sequences namely *encoder-outputs* (hidden states of all steps) and *hidden* (hidden state of the last step). At-Dec then takes both sequences to predict the target sequence. Both unidirectional and bidirectional encoders were used in experimentations. The *encoder-outputs* and *hidden* produced by the unidirectional encoder (Uni-Enc) have the dimensions of $[S-I \times B \times H]$ and $[1 \times B \times H]$ respectively where S = length of the source sequence, B = batch size, and H = hidden size. For a bidirectional encoder (Bi-Enc), the dimensions become $[S-I \times B \times 2H]$ and $[1 \times B \times 2H]$ respectively. The last dimensions are doubled because Bi-Enc produces forward and backward states. Since At-Dec is unidirectional, it expects *encoder-outputs* and *hidden* dimensions to be like those of Uni-Enc. Therefore, the forward and backward hidden states of Bi-Enc are added, so that the dimensions of Bi-Enc become like those of Uni-Enc. At first step of decoding, At-Dec *hidden* is initialized with encoder's *hidden*. In the following steps, At-Dec *hidden* is updated with its own *hidden* state. One more sequence that is taken by At-Dec as input is *decoder-inputs* which is either ground-truth (when trained with teacher forcing) or At-Dec prediction (when trained with sampling). The black circle in Fig.1 means concatenation and the asterisk means matrix multiplication. The part of the source code that is responsible for reading data is mostly adopted from [2]. The implementation of attention is adopted from [24].

V. EXPERIMENTS AND RESULTS

A. Experimental Setup

The experiments were conducted using Google Collaboratory [18] which comes with Python version 3.6.9 and PyTorch [24] version 1.4.0. In all experimentations, we use a learning rate of 0.05, a decay factor of 0.95 and a decay step of 10.000. The gradient is clipped to the L2 norm of 5. We chose the batch size to be 16 and trained for 100.000 iterations using stochastic gradient descent (SGD) optimizer with a momentum of 0.90. Both encoder and decoder have a single GRU layer with 1024 hidden units. The model is trained to predict 25 frames (1sec) given 50 frames (2sec) of motion.

B. Experimental Results

For a fair comparison, this work evaluates on the same 8 seed sequences that were used for evaluation by earlier works [2, 4, 13]. Our models are trained on exponential maps representation and the errors are computed in Euler angles representations.

TABLE I. MEANS SQUARED ERRORS

	Walking				Eating				Smoking				Discussion			
milliseconds	80	160	320	400	80	160	320	400	80	160	320	400	80	160	320	400
ERD [6]	0.93	1.18	1.59	1.78	1.27	1.45	1.66	1.80	1.66	1.95	2.35	2.42	2.27	2.47	2.68	2.76
LSTM-3LR [6]	0.77	1.00	1.29	1.47	0.89	1.09	1.35	1.46	1.34	1.65	2.04	2.16	1.88	2.12	2.25	2.23
SRNN [13]	0.81	0.94	1.16	1.30	0.97	1.14	1.35	1.46	1.45	1.68	1.94	2.08	1.22	1.49	1.83	1.93
Res. sup. [2]	0.28	0.49	0.72	0.81	0.23	0.39	0.62	0.76	0.31	0.68	1.01	1.09	0.33	0.61	1.05	1.15
AGED w/ adv+geo [1]	0.22	0.36	0.55	0.67	0.17	0.28	0.51	0.64	<u>0.27</u>	0.43	0.82	0.84	<u>0.27</u>	0.56	0.76	0.83
Uni-Enc-50	0.22	0.21	<u>0.24</u>	<u>0.24</u>	0.25	0.23	<u>0.20</u>	0.21	0.33	0.34	0.40	0.30	0.23	0.29	0.24	<u>0.25</u>
Bi-Enc-50	0.22	0.21	0.23	<u>0.24</u>	<u>0.19</u>	<u>0.20</u>	<u>0.20</u>	0.19	0.29	0.29	<u>0.38</u>	<u>0.26</u>	<u>0.25</u>	0.31	<u>0.25</u>	0.26
Uni-Enc-0	0.22	<u>0.22</u>	0.25	0.26	0.22	0.21	<u>0.20</u>	<u>0.20</u>	0.28	0.32	0.38	0.28	0.29	0.33	0.29	0.29
Uni-Enc-100	<u>0.23</u>	0.21	0.23	0.23	0.21	0.19	0.18	0.19	<u>0.31</u>	0.34	0.41	0.30	0.27	0.27	0.24	0.24
	Directions				Greeting				Phoning				Posing			
milliseconds	80	160	320	400	80	160	320	400	80	160	320	400	80	160	320	400
Res. sup. [2]	0.26	0.47	0.72	0.84	0.75	1.17	1.74	1.83	<u>0.23</u>	0.43	<u>0.69</u>	<u>0.82</u>	<u>0.36</u>	0.71	1.22	1.48
AGED adv+geo [1]	0.23	0.39	0.63	0.69	0.56	0.81	1.30	1.46	0.19	0.34	0.50	0.68	0.31	0.58	1.12	1.34
Uni-enc-50	0.25	0.23	<u>0.23</u>	0.20	0.28	0.27	0.29	0.30	0.52	<u>0.42</u>	0.86	0.86	0.42	0.37	0.36	0.33
Bi-enc-50	0.25	0.22	0.24	0.20	<u>0.31</u>	0.25	0.25	0.30	0.54	0.48	0.88	0.90	0.40	0.34	<u>0.31</u>	<u>0.32</u>
Uni-enc-0	<u>0.24</u>	0.23	0.22	0.19	<u>0.31</u>	<u>0.26</u>	<u>0.27</u>	<u>0.29</u>	0.53	0.47	0.87	0.88	0.38	<u>0.35</u>	0.33	0.34
Uni-enc-100	<u>0.24</u>	0.23	0.22	0.19	<u>0.31</u>	<u>0.26</u>	<u>0.27</u>	0.28	0.58	0.48	0.87	0.88	0.40	0.36	0.30	0.30
	Purchases				Sitting				Sitting down				Taking photo			
milliseconds	80	160	320	400	80	160	320	400	80	160	320	400	80	160	320	400
Res. sup. [2]	0.51	0.97	1.07	1.16	0.51	0.97	1.07	1.16	0.51	0.97	1.07	1.16	0.51	0.97	1.07	1.16
AGED adv+geo [1]	0.46	0.78	1.01	1.07	0.46	0.78	1.01	1.07	0.46	0.78	1.01	1.07	0.46	0.78	1.01	1.07
Uni-enc-50	0.42	0.38	0.34	0.34	0.42	0.38	0.34	0.34	0.42	0.38	0.34	0.34	0.42	0.38	0.34	0.34
Bi-enc-50	0.39	<u>0.33</u>	0.32	<u>0.29</u>	0.39	<u>0.33</u>	0.32	<u>0.29</u>	0.39	<u>0.33</u>	0.32	<u>0.29</u>	0.39	<u>0.33</u>	0.32	<u>0.29</u>
Uni-enc-0	<u>0.38</u>	0.32	<u>0.30</u>	0.28	<u>0.38</u>	0.32	<u>0.30</u>	0.28	<u>0.38</u>	0.32	<u>0.30</u>	0.28	<u>0.38</u>	0.32	<u>0.30</u>	0.28
Uni-enc-100	0.36	0.32	0.28	0.28	0.46	<u>0.43</u>	<u>0.44</u>	<u>0.43</u>	0.65	0.67	0.55	0.52	<u>0.26</u>	<u>0.27</u>	<u>0.24</u>	<u>0.23</u>
	Waiting				Walking dog				Walking together				Average			
milliseconds	80	160	320	400	80	160	320	400	80	160	320	400	80	160	320	400
Res. sup. [2]	0.28	0.53	1.02	1.14	0.28	0.53	1.02	1.14	0.28	0.53	1.02	1.14	0.28	0.53	1.02	1.14
AGED adv+geo [1]	0.24	0.50	1.02	1.13	0.24	0.50	1.02	1.13	0.24	0.50	1.02	1.13	0.24	0.50	1.02	1.13
Uni-enc-50	0.32	0.33	0.30	0.29	0.32	0.33	0.30	0.29	0.32	0.33	0.30	0.29	0.32	0.33	0.30	0.29
Bi-enc-50	0.29	0.28	<u>0.27</u>	<u>0.27</u>	0.29	0.28	<u>0.27</u>	<u>0.27</u>	0.29	0.28	<u>0.27</u>	<u>0.27</u>	0.29	0.28	<u>0.27</u>	<u>0.27</u>
Uni-enc-0	<u>0.27</u>	<u>0.29</u>	0.26	0.26	<u>0.27</u>	<u>0.29</u>	0.26	0.26	<u>0.27</u>	<u>0.29</u>	0.26	0.26	<u>0.27</u>	<u>0.29</u>	0.26	0.26
Uni-enc-100	0.30	0.30	<u>0.27</u>	0.28	0.30	0.30	<u>0.27</u>	0.28	0.30	0.30	<u>0.27</u>	0.28	0.30	0.30	<u>0.27</u>	0.28

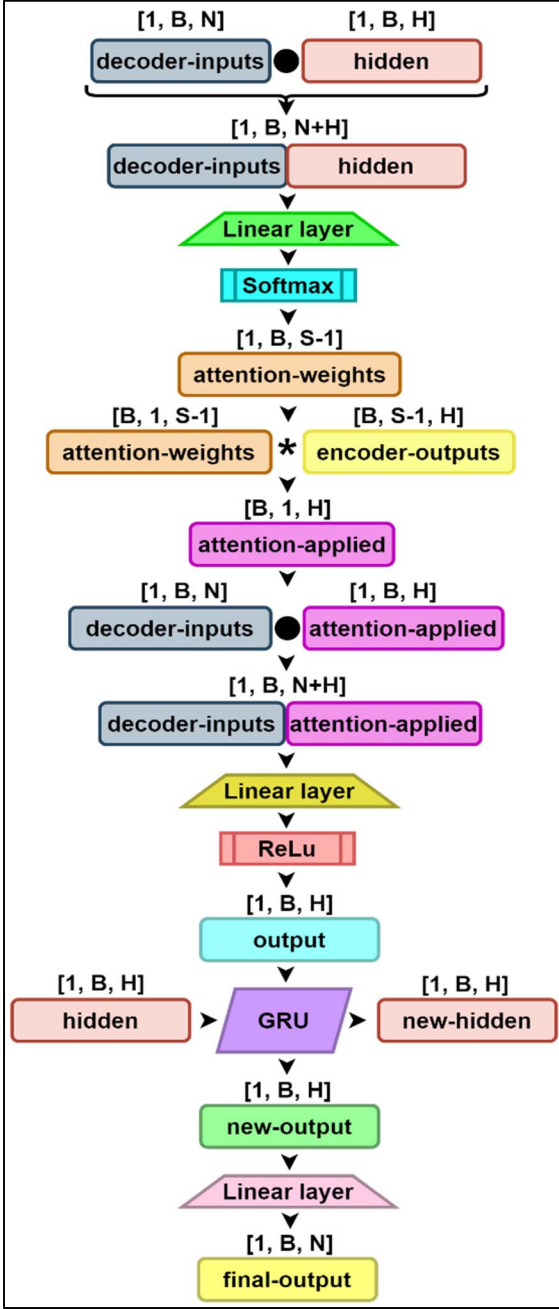


Fig. 1: Steps followed by the attention decoder used in this work. S =input sequence length, B = batch size, H =hidden size, and N =input size (No. features).

As in previous works, the frame-wise mean squared errors in Euler space are calculated between the prediction and the ground-truth for all 15 actions as shown in Table 1. The errors are computed after 80ms, 160ms, 320ms and 400ms of motion prediction i.e. errors are computed after 2, 4, 8 and 10 frames of prediction since the frame rate is 25 fps (data is down sampled by 2).

We have conducted 4 experimentations by designing the following 4 models:

- **Uni-Enc-50:** Unidirectional encoder with 50% teacher forcing.
- **Bi-Enc-50:** Bidirectional encoder with 50% teacher forcing.
- **Uni-Enc-100:** Unidirectional encoder with 100% teacher forcing.
- **Uni-Enc-0:** Unidirectional encoder with 0% teacher forcing.

All our models use the same attention decoder. As shown in Table 1 For walking, eating, smoking, and discussion actions, our models are compared against 5 earlier models. For the remaining actions, our models are compared against 2 earlier models (**Res. sup.** and **AGED**) since other models of earlier works haven't been evaluated on these actions. The 1st lowest error is shown in bold and the 2nd lowest error is shown underlined. As shown in Table 1, our models have comparable errors to **Res. sup.** [2] and **AGED** [1] after 80ms. However, as the time of prediction increases, the errors of earlier models deteriorate progressively unlike our models which show very stable prediction performance on all actions except phoning action. In fact, our models' performance stays as high even after 560ms and 1000ms. This demonstrates the suitability of our models not only for accurate motion prediction but also for accurate motion generation. On average, our models have beaten earlier models with a large margin after 160ms of prediction. According to the results of our models, changing the percentage of teacher forcing (the probability that the decoder sees ground-truth during training as opposed to seeing its own predictions) has no real impact on the results. Moreover, the use of a bidirectional encoder has no added benefit over its unidirectional counterpart. On the contrary of [2] and [5], residual connections and Quaternions have not improved our results but rather make them worse.

VI. CONCLUSION AND FUTURE WORK

Our contribution is threefold:

- 1) We use a simple yet powerful Seq2Seq model with attention decoder which outperforms earlier models both in short-term and long-term motion prediction tasks. To the best of our knowledge, our work is the first to train an attention decoder to predict 15 different categories of human motion. Our models have shown robust and stable performance even after 1000ms.
- 2) We prove quantitatively the suitability of attention decoder in the task of human motion prediction and generation.
- 3) We conducted experiments to evaluate quantitatively the performance of a unidirectional encoder vs a bidirectional encoder in the task of human motion prediction. We found that both perform equally.
- 4) We conducted experiments to evaluate quantitatively the impact of training an attention decoder on the task of human

motion prediction with 3 different training approaches namely pure teacher forcing, pure sampling and scheduled sampling (50% teacher forcing and 50% sampling). We found that different training approach has no real impact on the results.

Future work may include validating the predicted motion qualitatively, experimenting with different forms of attention, comparing the performance of LSTM against GRU, and using larger datasets of human motion.

REFERENCES

- [1] L.-Y. Gui, Y.-X. Wang, X. Liang and J. M. F. Moura, "Adversarial Geometry-Aware Human Motion Prediction," in *Computer Vision -- ECCV 2018*, Cham, 2018.
- [2] J. Martinez, M. J. Black and J. Romero, "On human motion prediction using recurrent neural networks," *CoRR*, vol. abs/1705.02445, 2017.
- [3] Y. Tang, L. Ma, W. Liu and W.-S. Zheng, "Long-Term Human Motion Prediction by Modeling Motion Context and Enhancing Motion Dynamic," *CoRR*, vol. abs/1805.02513, 2018.
- [4] D. Pavllo, D. Grangier and M. Auli, "QuaterNet: A Quaternion-based Recurrent Model for Human Motion," *CoRR*, vol. abs/1805.06485, 2018.
- [5] D. Pavllo, C. Feichtenhofer, M. Auli and D. Grangier, "Modeling Human Motion with Quaternion-based Neural Networks," *CoRR*, vol. abs/1901.07677, 2019.
- [6] K. Fragkiadaki, S. Levine, P. Felsen and J. Malik, "Recurrent Network Models for Human Dynamics," in *2015 IEEE International*
- [7] J. Butepage, M. J. Black, D. Kragic and H. Kjellstrom, "Deep Representation Learning for Human Motion Prediction and
- [8] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
- [9] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, vol. 45, pp. 2673-2681, 11 1997.
- [10] C. Ionescu, D. Papava, V. Olaru and C. Sminchisescu, "Human3.6M: Large Scale Datasets and Predictive Methods for 3D Human Sensing in Natural Environments," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, pp. 1325-1339, 7 2014.
- [11] C. Ionescu, F. Li and C. Sminchisescu, "Latent structured models for human pose estimation," in *2011 International Conference on Computer Vision*, 2011.
- [12] G. W. Taylor, G. E. Hinton and S. T. Roweis, "Modeling Human Motion Using Binary Latent Variables," in *Advances in Neural Information Processing Systems 19*, B. Schölkopf, J. C. Platt and T. Hoffman, Eds., MIT Press, 2007, pp. 1345-1352.
- [13] A. Jain, A. R. Zamir, S. Savarese and A. Saxena, "Structural-RNN: Deep Learning on Spatio-Temporal Graphs," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [14] S. Amidi and A. Amidi, "Recurrent Neural Networks cheatsheet Star," CS 230 - Recurrent Neural Networks Cheatsheet. [Online]. Available: <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks>. [Accessed: 14-Feb-2020].
- [15] J. Chung, Ç. Gülçehre, K. Cho and Y. Bengio, "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling," *CoRR*, vol. abs/1412.3555, 2014.
- [16] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, pp. 1735-1780, 1997.
- [17] K. Cho, B. Merriënboer, D. Bahdanau and Y. Bengio, "On the Properties of Neural Machine Translation: Encoder-Decoder Approaches," *CoRR*, vol. abs/1409.1259, 2014.
- [18] Colab.research.google.com. (2020). Google Colaboratory. [online] Available at: <https://colab.research.google.com/notebooks/intro.ipynb> [Accessed 15 Feb. 2020].
- [19] K. Cho, B. Merriënboer, Ç. Gülçehre, F. Bougares, H. Schwenk and Y. Bengio, "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," *CoRR*, vol. abs/1406.1078, 2014.
- [20] I. Sutskever, O. Vinyals and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," *CoRR*, vol. abs/1409.3215, 2014.
- [21] D. Bahdanau, K. Cho and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.
- [22] M.-T. Luong, H. Pham and C. D. Manning, "Effective Approaches to Attention-based Neural Machine Translation," *CoRR*, vol. abs/1508.04025, 2015.
- [23] S. Robertson, *NLP From Scratch: Translation with a Sequence to Sequence Network and Attention*.
- [24] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. Alché-Buc, E. Fox and R. Garnett, Eds., Curran Associates, Inc., 2019, pp. 8024-8035.